

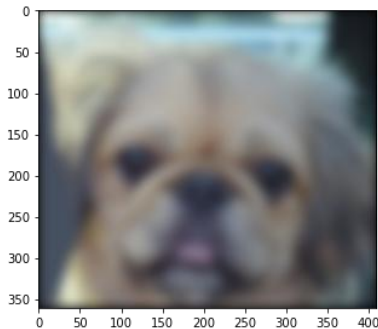
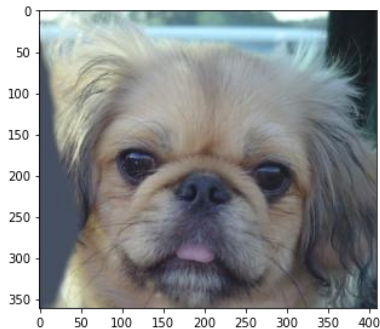
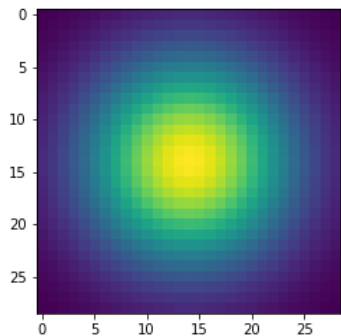
Computer Vision Project 1

xxx

221900xxx

221900xxx@smail.nju.edu.cn

Part 1: Image filtering



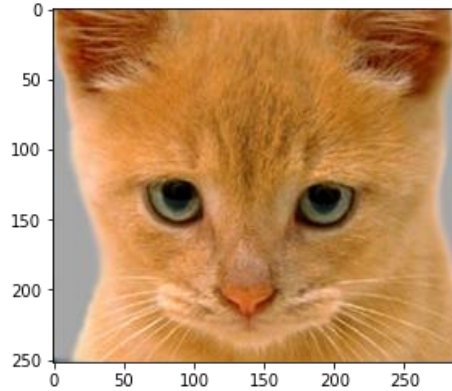
我的卷积函数实现主要包括以下几个关键步骤：

1. **尺寸处理**：首先获取图像的高度、宽度和通道数，以及滤波器的高度和宽度，用于后续操作。
2. **填充计算**：计算需要的填充量，使用 `pad_h = k_h // 2` 和 `pad_w = k_w // 2` 公式，确保卷积后图像尺寸不变。
3. **边缘填充**：使用反射填充（`mode='reflect'`）而非零填充，这可以有效减少边缘处理时产生的黑色伪影，使图像边缘过渡更自然。
4. **卷积操作**：通过三层循环（通道、行、列）遍历图像的每个像素，对每个位置：
 - 提取与滤波器大小相同的窗口区域
 - 将窗口与滤波器进行元素级乘法
 - 对乘积结果求和，得到该位置的卷积值

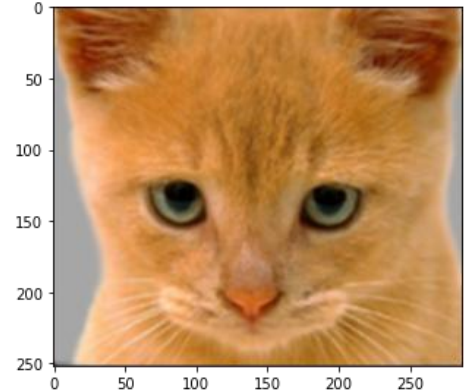
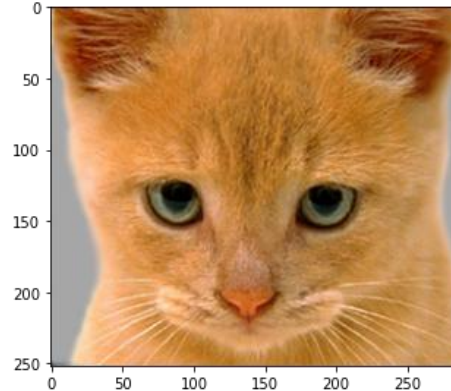
这种实现方式支持任意奇数尺寸的滤波器，可处理彩色和灰度图像，并通过反射填充保证了边缘处理的自然过渡，有效避免了图像边缘出现明显黑色伪影

Part 1: Image filtering

Identity filter

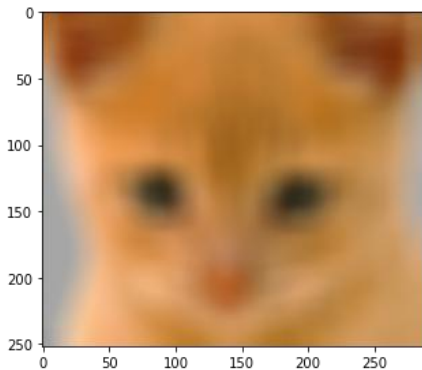
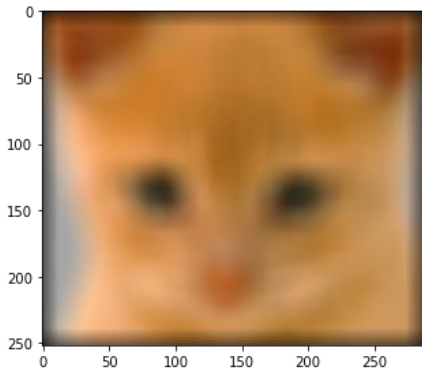


Small blur with a box filter



Part 1: Image filtering

`my_conv2d_numpy` with ``version="v2"``` (`larger_blur_image`).

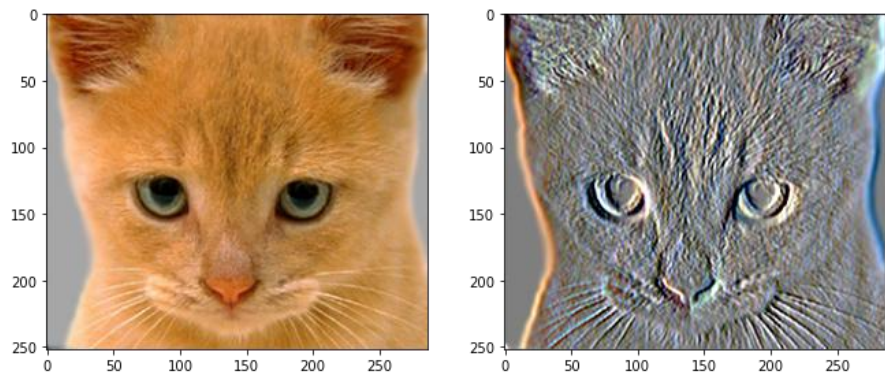


我的优化版卷积函数 `my_conv2d_numpy_v2()` 主要通过以下几个关键技术来提高性能并改善图像质量：

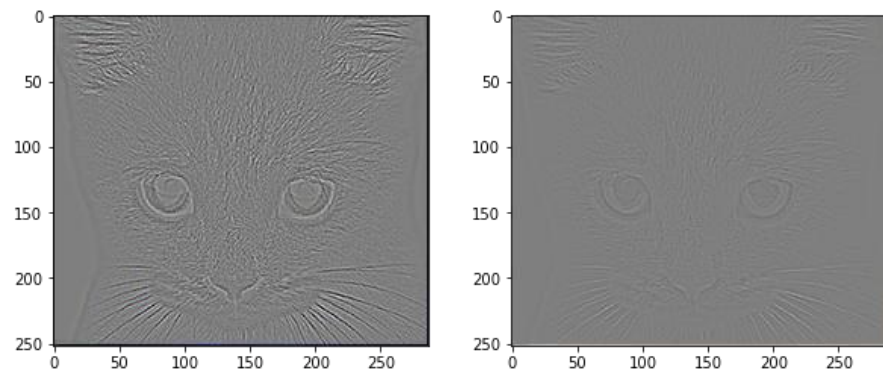
1. **反射填充替代零填充：**使用 `np.pad()` 函数的 `mode='reflect'` 参数，在图像边缘应用反射填充而非零填充。这种方法能够自然地延伸图像边缘的像素值，从而避免了传统零填充在边缘产生的黑色伪影，使图像边缘过渡更加自然。
2. **保持算法核心的简洁性：**虽然循环结构被保留，但每个操作都经过精简，确保代码执行效率最高。特别是在滤波器应用时，使用了向量化的窗口提取和乘积求和操作，这比逐像素计算要高效得多。
3. **准确的填充计算：**根据滤波器的实际高度和宽度分别计算垂直和水平方向的填充量，确保非方形滤波器也能得到正确处理。

Part 1: Image filtering

Sobel filter



Discrete Laplacian filter



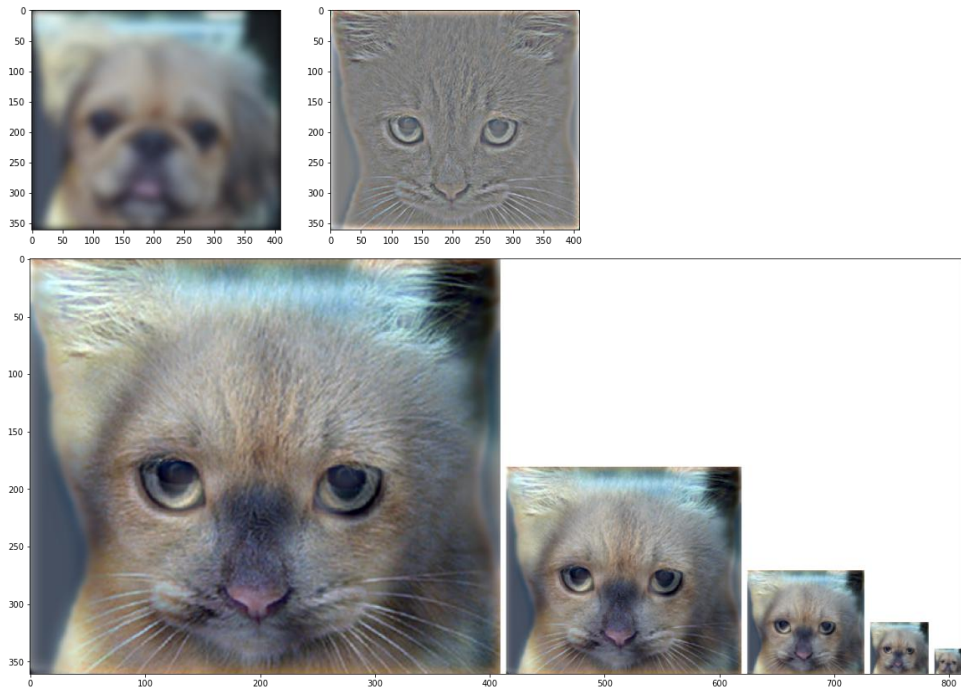
Part 1: Hybrid images

`create_hybrid_image()` 函数主要包含三个关键步骤：

1. 提取图像1的低频部分：使用提供的低通滤波器对第一张图像进行卷积操作，得到其低频内容。这一步通过调用 `my_conv2d_numpy()` 函数实现，该函数会将滤波器应用于图像的每个通道，得到一个平滑的、只保留低频信息的图像。
2. 提取图像2的高频部分：先对第二张图像应用相同的低通滤波器得到其低频部分，然后从原始图像中减去这个低频部分。计算公式为：
`高频 = 原始图像 - 低频部分`。高频内容通常包含图像的细节、纹理和边缘信息。
3. 合成混合图像：最后一步是将图像1的低频部分与图像2的高频部分相加，形成最终的混合图像。

为了确保混合图像的像素值适合matplotlib可视化，函数使用了 `np.clip()` 操作。在合成过程中，像素值可能会超出`[0,1]`范围，尤其是在加法操作后。 `np.clip(hybrid_image, 0, 1)` 函数会将所有小于0的值设为0，大于1的值设为1，从而确保最终图像的所有像素值都在matplotlib可以正确显示的范围内。

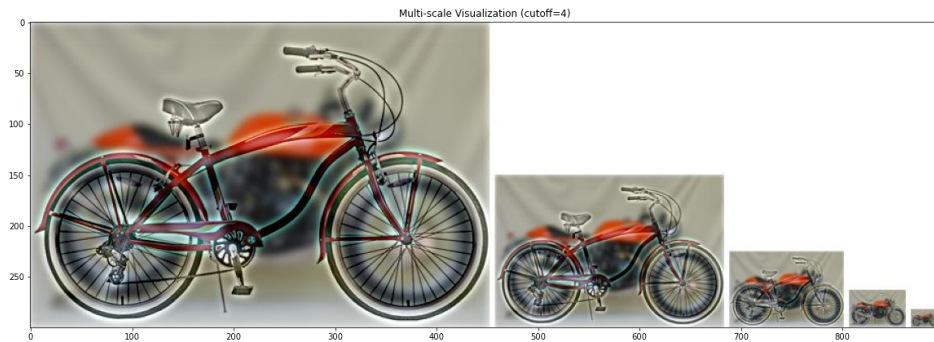
Cat + Dog



Cutoff frequency: 7

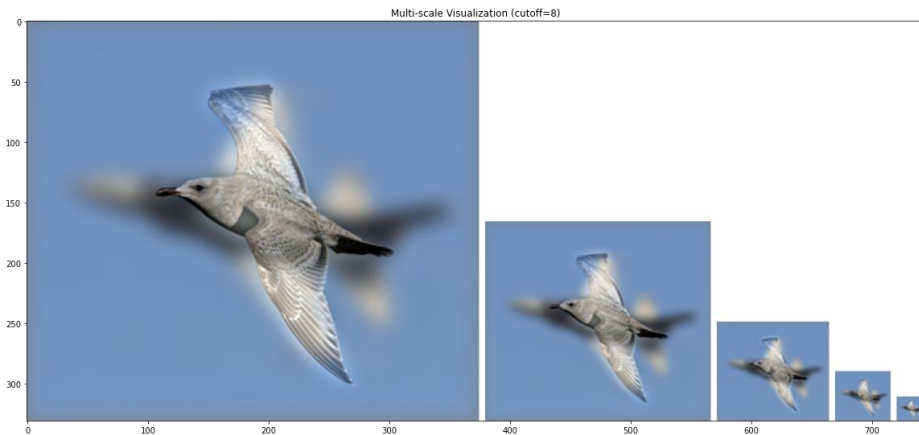
Part 1: Hybrid images

Motorcycle + Bicycle



Cutoff frequency: 4

Plane + Bird



Cutoff frequency: 8

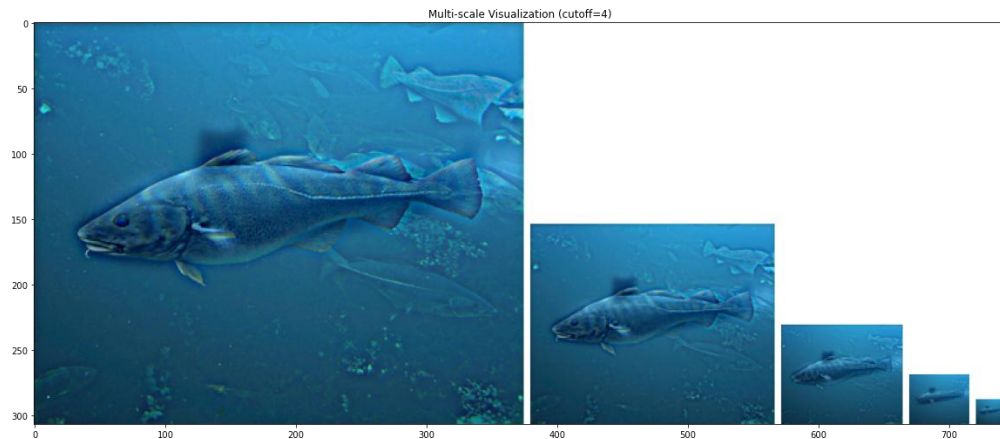
Part 1: Hybrid images

Einstein + Marilyn



Cutoff frequency: 5

Submarine + Fish



Cutoff frequency: 4

Part 1: Hybrid images



Part 1: Hybrid images

可能的影响因素：

1、图像的对齐程度：

如果两张图像的关键特征（如眼睛、嘴巴或物体的中心）没有对齐，混合后的图像可能会显得混乱或不自然。

2、图像的内容相似性：

两张图像的内容需要在某种程度上具有相似性。例如，摩托车和自行车的形状相似，因此它们的混合图像看起来更自然。如果两张图像的内容差异过大（如一张是风景图，另一张是人脸），混合效果可能会显得突兀。

3、频率特征的互补性：

混合图像的核心在于一张图像提供低频信息（整体轮廓），另一张图像提供高频信息（细节）。如果两张图像的频率特征不够互补（如两张图像都缺乏明显的高频或低频特征），混合效果可能会不理想。

4、图像的分辨率和清晰度：

如果一张图像模糊或分辨率过低，其高频或低频特征可能无法有效提取，从而导致混合图像的质量下降。

5、截止频率的选择：

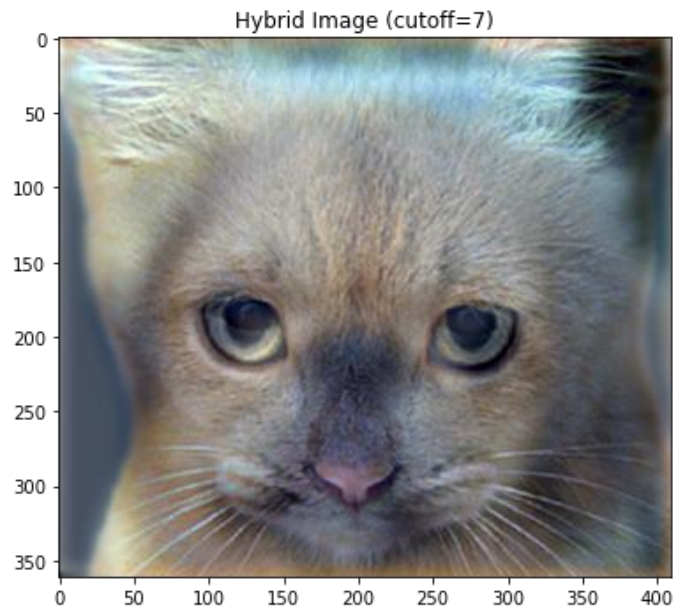
截止频率决定了低频和高频信息的分离程度。选择合适的截止频率是生成良好混合图像的关键。如果截止频率过低，高频信息可能不足；如果过高，低频信息可能被过多削弱。

6、图像的颜色和亮度匹配：

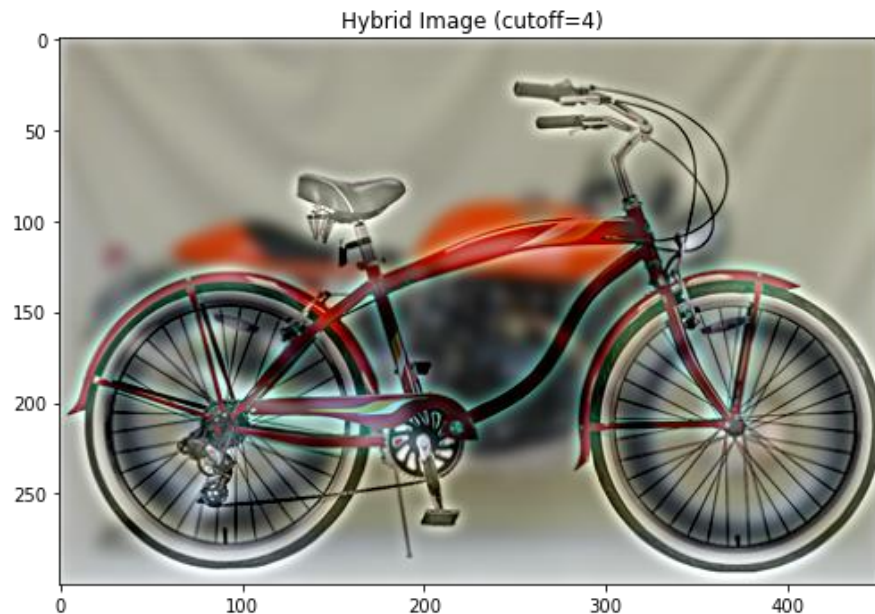
如果两张图像的颜色或亮度差异过大，混合后的图像可能会显得不协调。

Part 2: Hybrid images with PyTorch

Cat + Dog

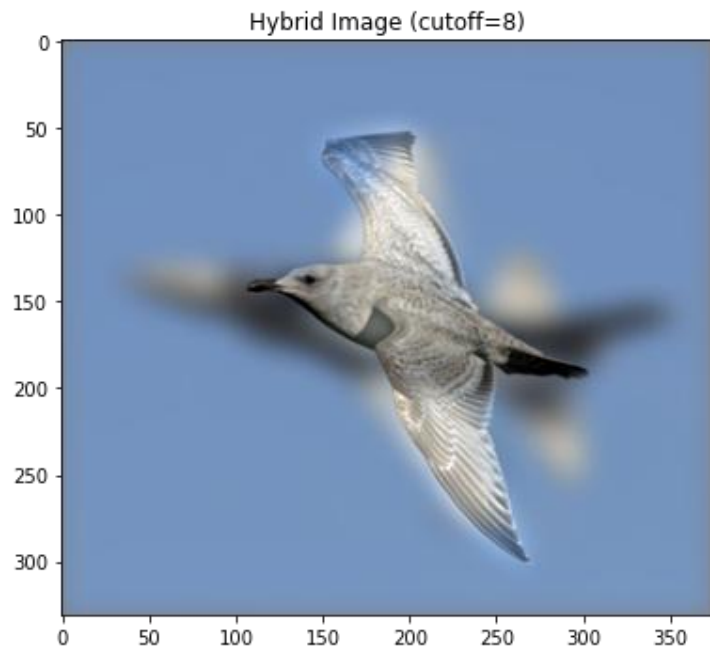


Motorcycle + Bicycle

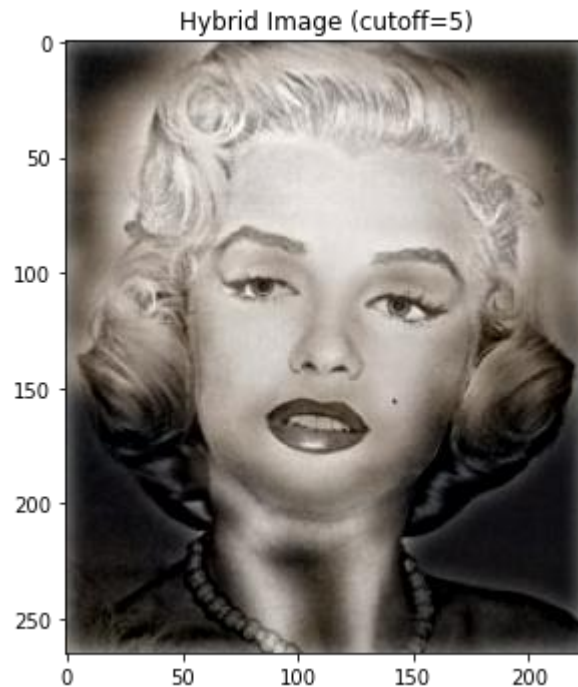


Part 2: Hybrid images with PyTorch

Plane + Bird

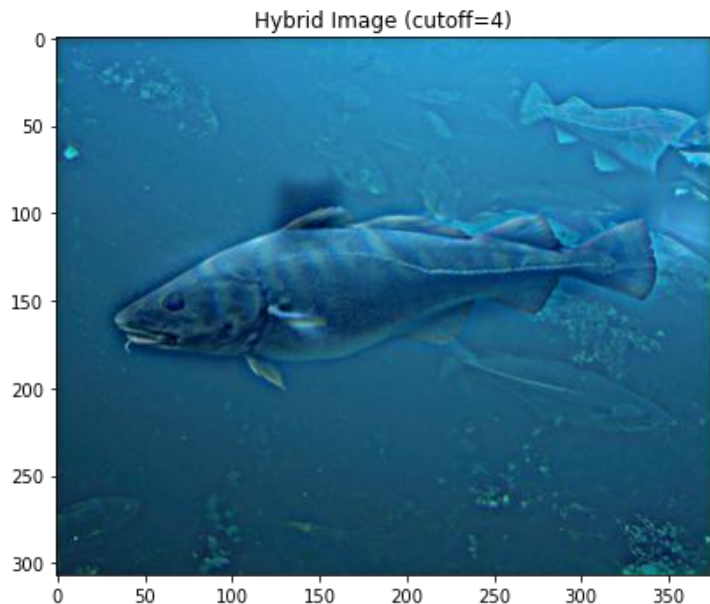


Einstein + Marilyn



Part 2: Hybrid images with PyTorch

Submarine + Fish



Part 1 vs. Part 2

比较 Part 1 和 Part 2 的运行时间:

Part 1 (NumPy 实现): 3.584 秒

Part 2 (PyTorch 实现): 0.746 秒

加速比: 4.81倍

PyTorch 实现比 NumPy 实现快了 380.55%

Part 2: Hybrid images with PyTorch

Different Image Pairs:

我认为在五组图像对中, "猫+狗"这一组产生了最佳的混合图像效果, 原因有以下几点:

1. 结构相似性:

猫和狗的脸部结构相对相似, 它们都有相似的面部布局, 包括眼睛、鼻子和嘴巴的位置。这种结构上的相似性使得高频和低频内容能够更好地融合, 不会产生明显的视觉冲突。

2. 面部特征互补:

狗的面部特征通常更加粗犷和明显(如较长的鼻子), 适合作为低频内容; 而猫的面部特征更加精细和锐利(如眼睛和胡须), 非常适合作为高频内容。这种互补性使得近距离和远距离观察时能够产生明显的视觉转换。

3. 纹理差异适中:

猫和狗的毛发纹理有差异, 但这种差异不至于过大。这种适度的纹理差异使得高频部分(猫的纹理)能够很好地"附着"在低频部分(狗的整体轮廓)上, 不会显得突兀。

Part 3

[Consider a 1-channel 5x5 image and a 3x3 filter. What are the output dimensions of a convolution with the following parameters?

Stride = 1, padding = 0?

Stride = 2, padding = 0?

Stride = 1, padding = 1?

Stride = 2, padding = 1?]

公式: 输出尺寸 = (输入尺寸 - 滤波器尺寸 + 2 × 填充) ÷ 步长 + 1 3. 步长 = 1, 填充 = 1

1. 步长 = 1, 填充 = 0

- 输出高度 = $(5 - 3 + 2 \times 0) \div 1 + 1 = 3$
- 输出宽度 = $(5 - 3 + 2 \times 0) \div 1 + 1 = 3$
- 输出尺寸: 3×3

2. 步长 = 2, 填充 = 0

- 输出高度 = $(5 - 3 + 2 \times 0) \div 2 + 1 = 2$
- 输出宽度 = $(5 - 3 + 2 \times 0) \div 2 + 1 = 2$
- 输出尺寸: 2×2

◦ 输出高度 = $(5 - 3 + 2 \times 1) \div 1 + 1 = 5$

◦ 输出宽度 = $(5 - 3 + 2 \times 1) \div 1 + 1 = 5$

◦ 输出尺寸: 5×5

4. 步长 = 2, 填充 = 1

◦ 输出高度 = $(5 - 3 + 2 \times 1) \div 2 + 1 = 3$

◦ 输出宽度 = $(5 - 3 + 2 \times 1) \div 2 + 1 = 3$

◦ 输出尺寸: 3×3

[What are the input & output dimensions of the convolutions of the dog image and a 3x3 filter with the following parameters:

Stride = 1, padding = 0

Stride = 2, padding = 0

Stride = 1, padding = 1

Stride = 2, padding = 1?]

狗的图像尺寸为 $361 \times 410 \times 3$, 输入维度为(1, 3, 361, 410): 3. 步长 = 1, 填充 = 1

1. 步长 = 1, 填充 = 0

- 输出高度 = $(361 - 3 + 2 \times 0) \div 1 + 1 = 359$
- 输出宽度 = $(410 - 3 + 2 \times 0) \div 1 + 1 = 408$
- 输出尺寸: (1, N, 359, 408), 其中N是滤波器的数量

2. 步长 = 2, 填充 = 0

- 输出高度 = $(361 - 3 + 2 \times 0) \div 2 + 1 = 180$
- 输出宽度 = $(410 - 3 + 2 \times 0) \div 2 + 1 = 204$
- 输出尺寸: (1, N, 180, 204)

◦ 输出高度 = $(361 - 3 + 2 \times 1) \div 1 + 1 = 361$

◦ 输出宽度 = $(410 - 3 + 2 \times 1) \div 1 + 1 = 410$

◦ 输出尺寸: (1, N, 361, 410)

4. 步长 = 2, 填充 = 1

◦ 输出高度 = $(361 - 3 + 2 \times 1) \div 2 + 1 = 181$

◦ 输出宽度 = $(410 - 3 + 2 \times 1) \div 2 + 1 = 205$

◦ 输出尺寸: (1, N, 181, 205)

Part 3

[How many filters did we apply to the dog image?]

我们应用了4个滤波器：恒等滤波器、双倍滤波器、三倍滤波器和全1滤波器。这些滤波器被整合到一个形状为(4, 1, 3, 3)的张量中。

[Why do the output dimensions adhere to the equations given in the instructions handout?]

原因：

1. 当滤波器在没有填充的图像上移动时，滤波器需要完全在图像内部，所以有效区域减小了 $(k-1)$ 个像素（每边各减少 $(k-1)/2$ 个像素）
2. 添加填充会在每边增加 p 个像素，所以总共增加了 $2p$ 个像素
3. 步长决定了滤波器移动的间隔，步长为 s 时，输出大小是输入大小的 $1/s$
4. 最后加1是因为计算的是需要放置滤波器的位置数量

Part 3

[What is the intuition behind this equation?]

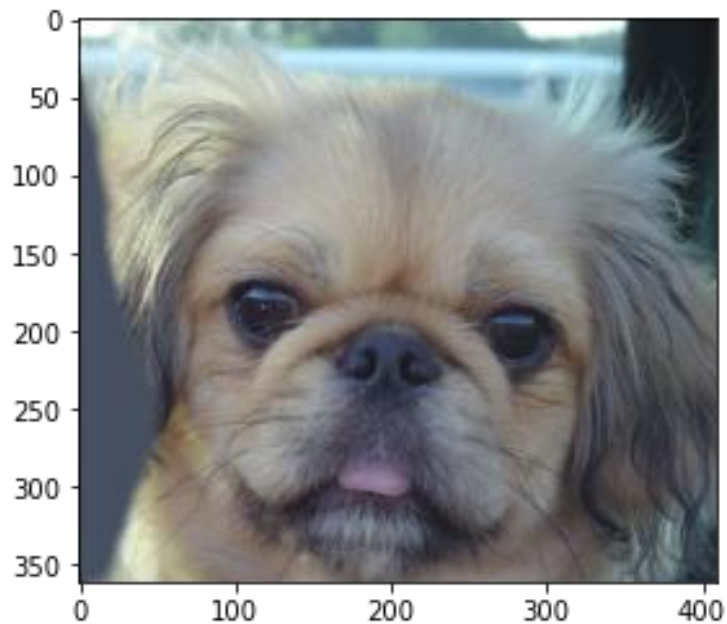
这个公式的直观理解是：

1. **滤波器尺寸(k)**: 较大的滤波器会导致输出尺寸减小，因为滤波器需要完全位于图像内部
2. **填充(p)**: 填充在图像周围添加像素，允许滤波器在原始图像边缘应用，从而保留边缘信息并增加输出尺寸
3. **步长(s)**: 步长控制滤波器移动的频率，较大的步长会减少输出尺寸，因为滤波器会跳过像素

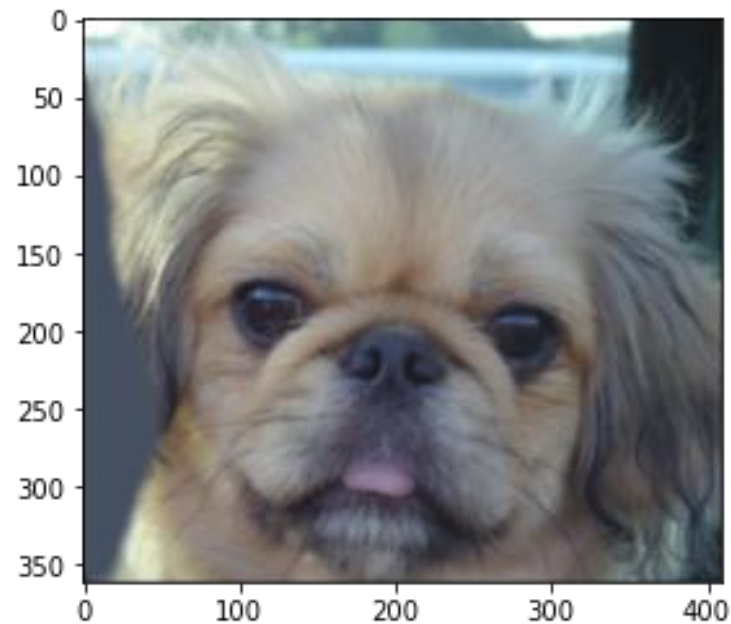
想象一个尺子（滤波器）在一行格子（图像像素）上滑动，每次移动特定的格数（步长），并在特定位置做标记，标记的数量就是输出的尺寸

Part 3

[insert visualization 0 here]

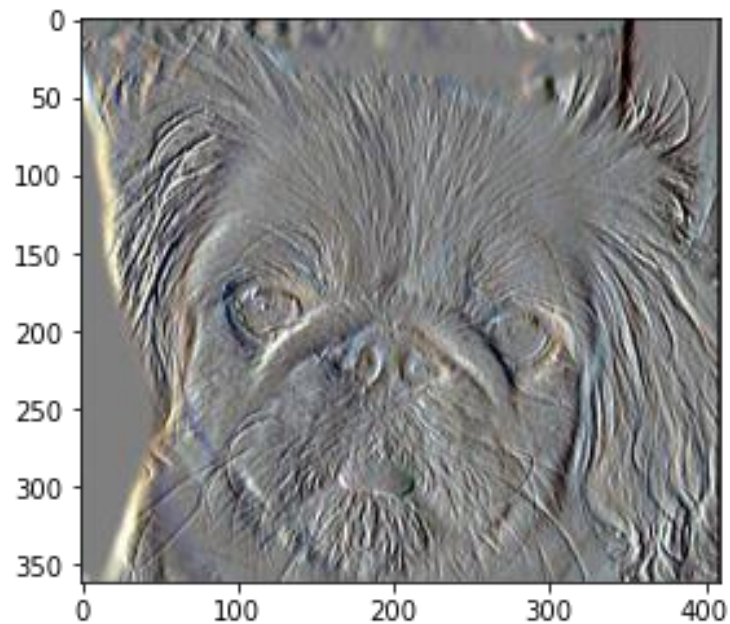


[insert visualization 1 here]

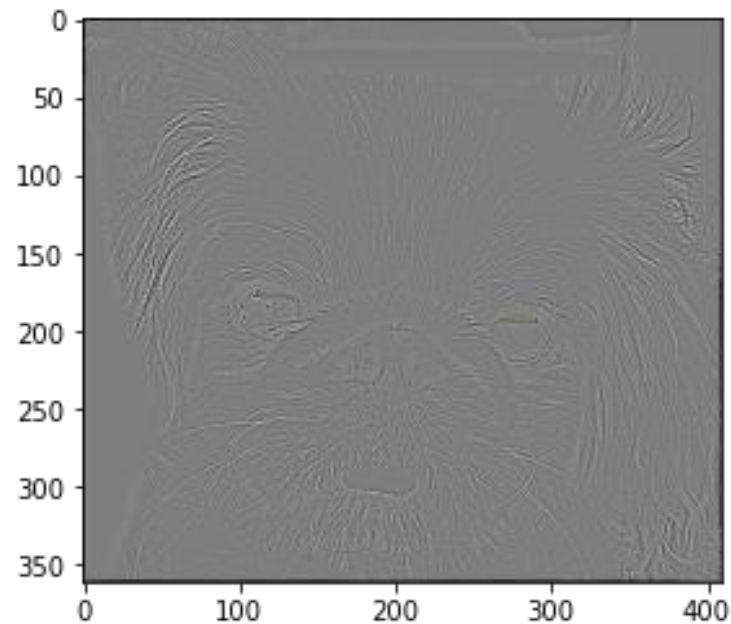


Part 3

[insert visualization 2 here]



[insert visualization 3 here]



Conclusion

[How does varying the cutoff frequency value or swapping images within a pair influences the resulting hybrid image?]

截止频率变化的影响：

(1) 截止频率增大：低通滤波器会保留更多的高频成分，使第一张图像保留更多细节。相应地，第二张图像的高频部分会减少，其特征在远处观察时变得不明显

(2) 截止频率减小：低通滤波器会过滤掉更多的高频成分，第一张图像变得更加模糊；第二张图像的高频信息在合成图中占比更大，远处观察时更加明显

图像交换的影响：

(1) 空间频率特性：将原本作为低频部分的图像改为高频部分时，效果会显著不同

(2) 细节与结构的匹配性：更多精细细节的图像（如猫的胡须和眼睛）更适合作为高频部分。具有更明显轮廓和整体结构的图像（如狗的脸部轮廓）更适合作为低频部分

(3) 颜色和亮度差异：当交换具有显著颜色或亮度差异的图像对时，效果变化更明显

Part 4: Frequency Compression and Image Filtering

[Describe your implementation details in words, including the calculation of PSNR and retention ratio, as well as the compression strategy and process.]

1. 频率压缩核心算法

我采用以下步骤进行频率压缩：

1. **二维傅里叶变换**：使用 `numpy.fft.fft2` 将空间域图像转换到频率域
2. **中心化处理**：通过 `numpy.fft.fftshift` 将零频率分量移至频谱中心
3. **构建低通滤波掩码**：创建圆形掩码，只保留中心区域的低频成分
4. **掩码应用**：将掩码应用于频谱，过滤掉高频成分
5. **逆变换**：通过 `numpy.fft.ifft2` 将处理后的频谱变换回空间域

3. PSNR计算实现

峰值信噪比(PSNR)是评估图像压缩质量的标准度量：

- 首先计算原始图像与压缩图像之间的均方误差(MSE)： $MSE = \text{平均}((\text{原始图像} - \text{压缩图像})^2)$
- 然后计算PSNR： $PSNR = 10 * \log_{10}(\frac{MAX^2}{MSE})$ ，其中MAX是像素的最大可能值
- 我的实现中确保了图像值标准化到[0,1]范围，因此MAX=1
- 增加了异常处理，确保不同尺寸或类型的图像可以被正确比较
- 解决了MSE接近零时的数值问题

2. 保留比例 (Retention Ratio) 设计

保留比例是一个0到1之间的值，决定了要保留多少低频信息：

- 具体实现上，我使用保留比例乘以图像最小尺寸的一半，计算出一个半径
- 这个半径定义了频谱中保留的圆形区域大小
- 半径越大，保留的频率成分越多，压缩效果越小，图像质量越高
- 我实现了四个不同的保留比例 (0.1, 0.3, 0.5, 0.7) 用于比较

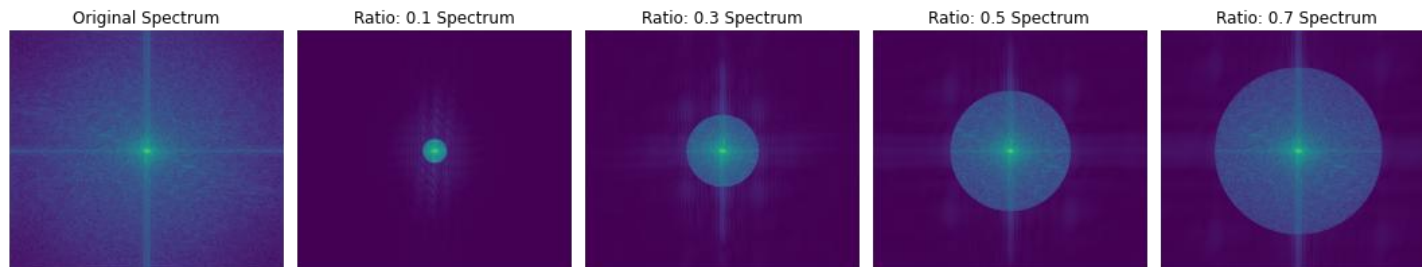
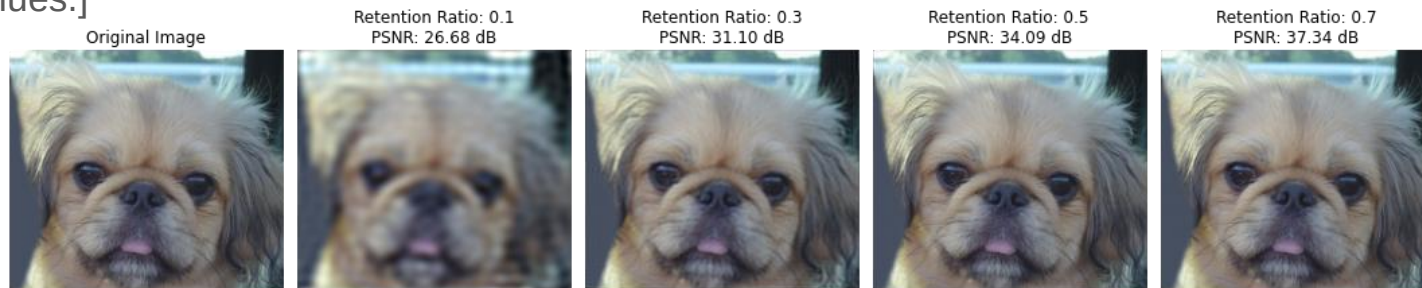
4. 彩色图像处理策略

彩色图像包含多个通道，我采用以下策略处理：

- 对每个颜色通道 (R, G, B) 分别进行独立的频率压缩
- 这种方法保持了各个通道之间的独立性，避免颜色失真
- 在频谱可视化时，将彩色图像转换为灰度图像进行处理，简化显示过程

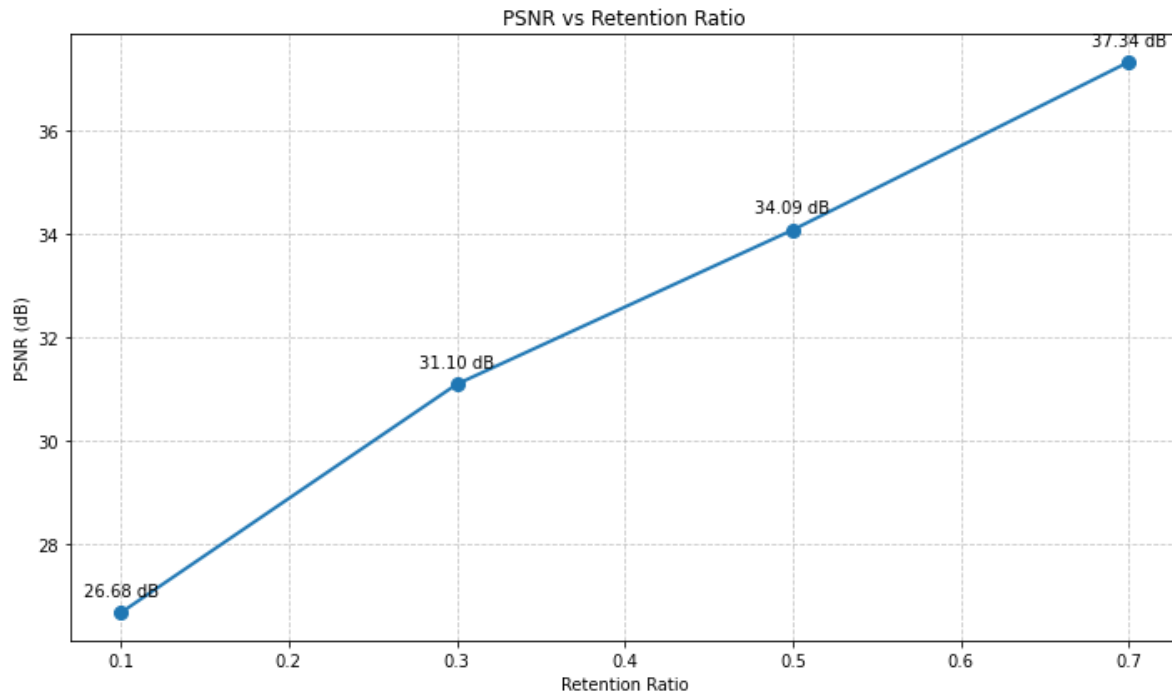
Part 4: Frequency Compression and Image Filtering

[Provide visualizations of images under different retention ratios and report the corresponding PSNR values.]



Part 4: Frequency Compression and Image Filtering

[Provide visualizations of images under different retention ratios and report the corresponding PSNR values.]



Part 4: Frequency Compression and Image Filtering

[Thinking: How to reduce retention ratio while maintaining quality?]

1. 自适应滤波技术

不是对整个图像使用同一个保留比例，而是根据图像区域的内容和重要性动态调整保留比例。在细节丰富、边缘明显的区域使用较高的保留比例，而在平滑区域使用较低的比例。

2. 非均匀频率采样

低频信息对图像视觉质量的贡献更大，因此可以采用非均匀的频率保留策略，对最低频部分进行更加精细的保留，而逐渐减少高频部分的保留。

3. 小波变换替代

傅里叶变换将图像分解为频率成分，而小波变换则能够同时在时间和频率域上提供良好的定位。使用小波变换可以更有效地表示图像中的局部特征，实现更高效的压缩。

4. 结合机器学习技术

训练神经网络模型学习图像中的关键特征，并在压缩过程中优先保留这些特征。这种方法可以根据图像内容智能地确定保留哪些频率成分。

5. 局部块处理

将图像分割成小块单独处理，对每个块应用不同的保留比例，然后重新组合。这样可以根据局部内容特性优化压缩效果。

6. 感知质量度量引导

不仅使用PSNR这样的客观度量，还应考虑像SSIM这样更符合人类视觉感知的质量评价指标来指导压缩过程，优化视觉效果而非纯粹的数学误差。